§ WORKING PAPER

EncodeBench

Measuring whether language models can write law as code, and what it takes to measure that honestly

Max Ghenis

PolicyEngine · Axiom Foundation

Revision 1 · 2026-08-07

Abstract

EncodeBench measures whether frontier language models can encode statutes into RuleSpec, the Axiom Foundation's format for computable law: executable rules that cite their sources, carry their effective dates, and ship with tests. Sixteen UK provisions, resolved from a signed corpus release, supply the tasks; each encoding grades on deterministic gates — an artifact exists, it compiles, it passes the repository's validation including the encoding's own companion tests, and its numeric literals — dates and small structural integers exempt — must appear in the source text. A gate pass certifies a mergeable, self-consistent, grounded draft, not legal correctness. Three boards to date cover six models. This paper reports the boards and, with equal weight, what running them taught about measurement: scores moved with the toolchain that produced them, so a score is citable only against its encoder version; a hardcoded timeout graded one model's slowness as failure; the harness gave different model families different exams; and a reasoning-effort setting that appeared to control the comparison did nothing at all. The first two surfaced through the same deterministic machinery that grades the models; the others took a read-only audit and a receiver probe, both posted in the open. Board v4, pre-registered here, runs on a parity encoder built to close these gaps, reporting means with spread in place of single draws.

STATUS

Working paper. Boards v1–v3 as posted to axiom-encode#1189; board v4 is pre-registered in § "Board v4, pre-registered."

Table of contents

1 Introduction 2

2 The task 3

3 Four gates and a fold 4

4 Three boards 5

5 What it takes to measure this honestly 8

5.1 Scores bind to the toolchain . . . 8

5.2 A timeout that graded slowness as failure . . . 9

5.3 The effort setting that did nothing . . . 9

5.4 Three backends, three exams . . . 10

5.5 Case 04, flagged and retracted . . . 10
5.6 Integrity notes . . . 11

6 Board v4, pre-registered **11**
6.1 The admission score . . . 12

7 Limitations **13**

8 Reproducibility **13**

References **14**

1 Introduction

The Axiom Foundation publishes statutes as executable code. An AI encoder drafts each encoding in RuleSpec — parameters and formulas that cite their statutory authority, carry their effective dates, and ship with companion tests — and a battery of deterministic checks gates what merges. That design turns model choice into an engineering question with money and correctness attached: which model drafts law-as-code well enough to sit inside a production pipeline, and how would you know?

EncodeBench makes the question measurable. It formalizes the informal bake-off the Axiom Foundation ran in July 2026 to route its own encoder, replacing a scratchpad comparison with a pinned suite, a deterministic fold, and public records; the fold aggregates runs into a board — the published comparison. Code benchmarks grade models on tests (Chen et al. 2021; Jimenez et al. 2024); legal benchmarks grade reasoning over statutes (Guha et al. 2023; Holzenberger et al. 2020); statute-to-structure and statute-to-code translation grades one-shot generation by similarity or human comparison (Janatian et al. 2023; Lorenzo et al. 2025); and the target formats exist as domain-specific languages for law (Mérigoux et al. 2021), deployed rules-as-code engines (OpenFisca contributors 2026), and government rules-as-code programs (Mohun and Roberts 2020). EncodeBench differs in what it grades and how: CLI encoder products graded by production merge gates rather than similarity-scored one-shot generation — a contrast boards v1–v3 realized fully only on the codex path (Section 5.4) — on a signed corpus release rather than pasted statute text. Its sister benchmark PolicyBench (Ghenis 2026) asks whether a model can compute the law; EncodeBench asks whether it can write it.

Three boards in, the results worth the most attention concern the measurement itself. Between two full-roster runs a day apart, most of the roster moved, and the movement skewed positive — so a single board reads as one sample, not a ranking. A hardcoded ceiling on one encoder path graded a slow model's unfinished cases as failures. A read-only audit found the three encoder backends administering different exams. And the setting that appeared to pin reasoning effort did nothing on any backend. Two of these four surfaced through the same deterministic machinery that grades the models — the fold's refusal to blend toolchains, and timeout evidence in the run records; the other two took a read-only audit and a receiver probe, both posted in the open. The paper reports the boards together with everything known to weaken them, then pre-registers the run that removes the weaknesses.

Two stakes belong on the table before any results. The benchmark routes the Axiom Foundation's own production encoder, which defaulted to gpt-5.6-terra when these boards ran — the boards exist to test that routing, and the terra-below-sol result of Section 5.1 runs against the default. And the author works at both the Axiom Foundation and PolicyEngine, whose UK model is the planned oracle — the independent implementation whose computed results would cross-check the encodings — for three of the sixteen cases.

2 The task

One case gives one model one provision. The harness resolves the provision's text from a signed corpus release — every run here pins uk-rulespec-2026-07-14, verified against its publishing key, with a recorded expression date (the consolidation date of the resolved text) per provision, 2026-04-06 to 2026-07-01 across the v3 suite — and places it in a cold workspace: the resolved source text and its provision metadata, and nothing else. The per-case workspace context manifests, frozen in this paper's snapshot, list zero context files for every cold case: no existing encoding of the target, no repository context, no stub files. Every board row that records a workspace binds to its case's manifest by hash; the five rows the harness killed before a workspace existed carry none. The model — throughout this paper, a *runner*: a CLI coding product wrapping a model, with its own system prompt, tools, and retry behavior — writes the RuleSpec module: the parameters and formulas the provision defines, citations back to it, effective dates, and the companion tests the repository requires of every rule. The suite covers UK law because the UK repository was the first to reach the signed-release toolchain the eval requires; a US suite follows once the US repository completes the same migration.

Capability cases run cold for a reason. A workspace with repository context includes whatever encoding of the provision already merged, so a capability score would come to measure copying, and would drift as the repository grows. The last three cases run exactly that way, as oracle candidates: their frozen manifests list 2–16 context files each, and every one includes the already-merged encoding of the target provision and its tests. Their cells therefore measure work with the merged answer available — the risk the cold design exists to avoid — and the board table carries a cold-only subscore beside the headline for that reason. They sit on the PolicyEngine UK comparison path and will carry a live oracle check once that plumbing is verified for eval cases; until then they grade on the same deterministic gates as everything else.

Table 1: The sixteen cases. Cases 1–13 run cold; cases 14–16 run with repository context as oracle candidates. The provision column derives from the citation path frozen in every run row. SSCBA = Social Security Contributions and Benefits Act; ITEPA = Income Tax (Earnings and Pensions) Act.

#	case	provision	probes	group
01	`vat_standard_rate`	Value Added Tax Act 1994 s. 2	flat-rate control case	parameters
02	`class_2_nic`	SSCBA 1992 s. 13	voluntary flat-rate contribution — see the labeling note below	parameters
03	`class_1_secondary_nic`	SSCBA 1992 s. 9	employer-side rate and secondary threshold	parameters
04	`income_tax_rate_bands`	Income Tax Act 2007 s. 10	basic, higher, and additional rate bands	bands
05	`dividend_nil_rate`	Income Tax Act 2007 s. 13A	band-conditional nil-rate amount	bands
06	`personal_allowance_taper`	Income Tax Act 2007 s. 35	£100,000 half-excess taper with round-up	tapers
07	`hicbc_phaseout`	ITEPA 2003 s. 681B	charge liability above the £60,000 threshold — see the labeling note below	tapers
08	`pension_annual_allowance`	Finance Act 2004 s. 228	annual allowance beside the s. 228ZA taper	tapers

#	case	provision	probes	group
09	`marriage_allowance_transfer`	Income Tax Act 2007 s. 55B	transferable allowance: elections, both-party conditions	structure
10	`income_tax_liability_steps`	Income Tax Act 2007 s. 23	multi-step liability calculation; import-heavy	structure
11	`uc_award_calculation`	Welfare Reform Act 2012 s. 8	universal credit: maximum amount minus reductions	structure
12	`child_benefit_entitlement`	SSCBA 1992 s. 141	entitlement only — the rates live in a separate instrument	grounding
13	`finance_act_2026_charge`	Finance Act 2026 s. 1	recency probe: a one-sentence charge for 2026–27	grounding
14	`income_tax_main_rates`	Income Tax Act 2007 s. 6	the main-rate structure (the percentages live in annual finance acts)	oracle path
15	`class_1_primary_nic`	SSCBA 1992 s. 8	employee-side Class 1 contributions	oracle path
16	`child_benefit_weekly_rates`	Child Benefit (Rates) Regulations 2006 reg. 2	the weekly rates case 12 must not invent	oracle path

Three suite-manifest statements misdescribe the instrument, and the paper describes the instrument rather than the statements. The case named `class_2_nic` pins a citation path that resolves to SSCBA 1992 s. 13, *Class 3* contributions — a voluntary flat-rate contribution with no small-profits threshold — so its name states the wrong contribution class; the suite manifest's own comment repeats the error. The high income child benefit charge (HICBC) case, `hicbc_phaseout`, resolves the liability and threshold provision (s. 681B); the taper arithmetic lives in s. 681C, which the workspace does not supply, so the case probes the charge condition, not the phaseout formula its name suggests — of the tapers group, only `personal_allowance_taper`'s workspace contains the taper arithmetic itself. And the manifest's header claims cold workspaces carry definition stubs; the frozen context manifests show none did. The case labels are frozen into the case identities of every board, so renaming them starts a new board; all three defects are filed upstream (axiom-encode#1438), and the measurements themselves stand — every model received the same resolved text, whatever the label said.

Two cases exist to catch invented numbers. `child_benefit_entitlement` supplies entitlement conditions whose weekly amounts live in a different instrument — the Child Benefit (Rates) Regulations 2006, SI 2006/965, which is case 16's provision — so any rate literal in that encoding is an invention. `finance_act_2026_charge` supplies an act that postdates most of the roster's training data; its resolved text is one sentence — the income tax charge for 2026–27 — so the case checks that a model encodes the supplied provision rather than a remembered template, and it carries no rate for recall to corrupt. A recency case whose parameter values changed post-cutoff, so that recall produces wrong numbers, belongs in a future suite revision.

3 Four gates and a fold

A case passes for a model only when four deterministic checks pass: the encode returns an artifact, the artifact compiles in the Axiom rules engine, repository validation passes — including the com-

panion tests — and the grounding scan finds zero numeric literals absent from the source text. The gate-pass count is the headline, and nothing else folds into it.

What a gate pass certifies deserves one plain statement. Compile and repository validation run independently of the model, but the companion tests are the model's own: a model that writes weak tests passes its own encoding more easily than one that writes strict tests. The grounding scan runs one-directional: an invented number fails it, an omitted number cannot. (The validator parses the artifact's RuleSpec, extracts the numeric literals its rules carry — exempting dates, structural index literals, the small integers –1 through 3, half-up-rounding helpers, and decimal-scale powers of ten — and matches each against the supplied source text's numeric occurrences, word forms included: passing taper artifacts carry 0.5 as a grounded literal against source text that says "one-half". Each decision records in the run artifact.) The omitted-number direction has no deterministic bound on these boards, and the advisory coverage column does not supply one: its denominator is the numeric occurrences of the artifact's own embedded source summary, not the supplied text, so a model that omits a number from both its encoding and its summary escapes the column entirely. On v3 it read 93–100% for five runners and 71.2% for opus-5 — a self-consistency reading, not source recall. A source-relative recall metric joins the v4 instrument list. A gate pass therefore certifies a mergeable, self-consistent, grounded draft — not a legally correct encoding, and not a complete one.

Everything else reports alongside as advisory columns. A pinned reviewer model scores each encoding's statutory fidelity; one reviewer model scores every runner's artifacts. The pin is the `opus` CLI alias, which resolved to claude-opus-4-8 under live trace verification on the v2 run day; v3 ran the same rig configuration the next day without an independent re-check, and the result rows do not record the resolved identity. A single noisy judge whose own family competes on the board cannot referee it neutrally, so its score sits beside the gates rather than inside them. On board v3 its mean scores span 7.8 to 8.3 of 10 across all six runners (scoring 15–16 of each runner's cases) — a band too narrow to order the leaders. The oracle column stays empty until the PolicyEngine UK runtime path is verified for eval cases. The median-latency and cost columns carry measured values.

The roster carries a control: `gpt-5.6-luna`, which prior internal measurement found weak at exactly this task. Across the three boards it scored 5, 5, 8 of 16 — at or near the bottom every time. A board that failed to separate the control from the leaders would indict the instrument, not the models. The control's certification has a scope, though: luna rides the codex backend, so it can catch defects on that path and is structurally blind to defects on the Claude path — and the two defects this paper reports on the Claude path (Section 5.2, Section 5.4) were exactly the kind it could not have caught.

Runs fold into one board only when the suite name, the ordered case identities, the corpus release, and the execution identity — the recorded, normalized set of score-affecting toolchain components — all match. Runner sets may differ: a model added later folds cleanly without re-running incumbents. Runs from a changed encoder refuse to fold, so a new toolchain starts a new board by construction. The variance finding (Section 5.1) and the timeout finding (Section 5.2) both end in that refusal doing real work.

4 Three boards

Table 2: The three boards. Costs are API-equivalents: codex-backend tokens priced at published July 2026 rates; Claude-backend costs are the CLI's own recorded per-call figures. All runs drew on subscription-billed seats.

board	run	encoder	runners	suite cost
v1	2026-07-23	`0.2.1333`	4	$15.11
v2	2026-07-25	`0.2.1380`	6	$40.25
v3	2026-07-26	`0.2.1382`	6	$52.09

Board v1 piloted the suite on four codex-backend models. Boards v2 and v3 ran the full six-model roster one day apart on successive encoder versions — v2 on the encoder that carried the timeout defect of Section 5.2, v3 on the encoder that fixed it. v3 stands as the current board.

Table 3: Board v3 — encoder 0.2.1382, run 2026-07-26. The gate-pass column counts cases that clear all four gates; cold subsets the thirteen cold cases (the three oracle-path cases ran with the merged target encoding in-workspace, Section 2). T counts cases that exhausted the recorded time budget; artifacts counts cases that produced a gradeable artifact; compile, ci (repository validation, companion tests included), and grounded rates cover produced artifacts only; median is case duration. OpenAI-family runners rode the codex path (workspace, tools) and Anthropic-family runners rode the Claude CLI prompt-only, so cross-family comparisons inherit Section 5.4. The advisory reviewer and oracle columns live in the board record.

runner	model	gate pass	cold	T	artifacts	compile	ci	grounded	median
gpt-5.5	`gpt-5.5`	**15/16** (93.8%)	13/13	—	16/16	100.0%	93.8%	100.0%	48s
sol	`gpt-5.6-sol`	**14/16** (87.5%)	11/13	—	16/16	100.0%	87.5%	100.0%	45s
fable	`claude-fable-5`	**12/16** (75.0%)	10/13	1	15/16	100.0%	80.0%	100.0%	266s
terra	`gpt-5.6-terra`	**11/16** (68.8%)	8/13	—	16/16	100.0%	68.8%	100.0%	34s
luna	`gpt-5.6-luna`	**8/16** (50.0%)	6/13	—	15/16	93.3%	53.3%	100.0%	50s
opus-5	`claude-opus-5`	**6/16** (37.5%)	5/13	—	16/16	87.5%	37.5%	100.0%	47s

Table 4: Board v3 per-case grid. P = gate pass, F = failed a gate (compile, validation, or grounding), T = timeout, E = no artifact. Cross-family column comparisons inherit Section 5.4.

case	gpt-5.5	sol	fable	terra	luna	opus-5
01 `vat_standard_rate`	P	P	P	P	P	F
02 `class_2_nic`	P	P	P	P	F	P
03 `class_1_secondary_nic`	P	P	P	P	E	F
04 `income_tax_rate_bands`	P	F	P	F	F	F
05 `dividend_nil_rate`	P	F	P	F	P	F
06 `personal_allowance_taper`	P	P	F	F	F	F
07 `hicbc_phaseout`	P	P	P	P	F	P
08 `pension_annual_allowance`	P	P	F	F	P	P
09 `marriage_allowance_transfer`	P	P	T	F	F	F
10 `income_tax_liability_steps`	P	P	P	P	F	F
11 `uc_award_calculation`	P	P	P	P	P	P
12 `child_benefit_entitlement`	P	P	P	P	P	P
13 `finance_act_2026_charge`	P	P	P	P	P	F
14 `income_tax_main_rates`	P	P	P	P	P	F
15 `class_1_primary_nic`	F	P	F	P	P	F
16 `child_benefit_weekly_rates`	P	P	P	P	F	P

Three readings of the current board survive the variance analysis of Section 5.1; where a reading crosses model families it also inherits the weaker evidentiary standard of Section 5.4, flagged in place.

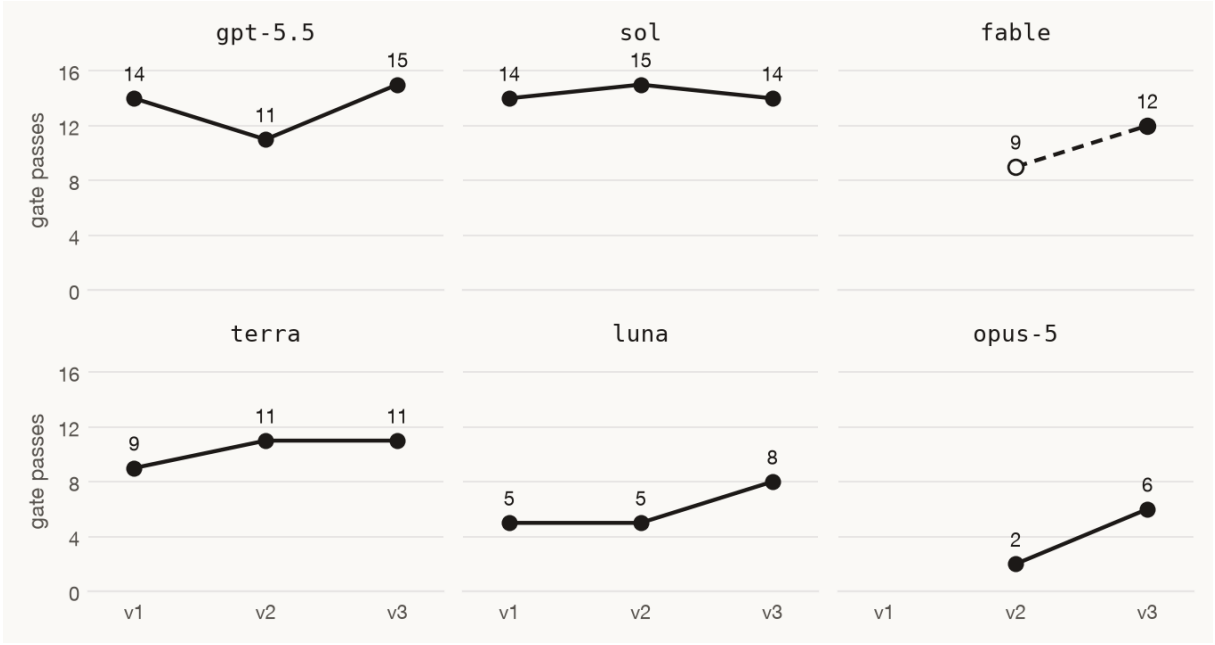


Figure 1: Gate passes per model across the three boards. Panels follow board v3 rank; the y-axis runs 0–16 cases. The boards ran on different encoder versions, so each line connects runs of the suite, not draws from one instrument — and fable and opus-5 rode the Claude CLI prompt-only while the rest rode the codex path with a workspace, so cross-family comparisons inherit Section 5.4. The hollow marker is board v2's fable row, which a 600-second harness ceiling truncated (Section 5.2) — a floor, not a capability reading. fable and opus-5 joined the published roster at v2; a completed same-toolchain fable run under the v1 encoder was never folded, frozen in this paper's snapshot as excluded (Section 5.6).

First, the extremes separate. Within the codex backend, gpt-5.5 and sol clear 15/16 and 14/16 while the control (luna) sits at 8/16; the smallest leader-to-tail gap on the board (6 cases) exceeds the largest gate-pass delta the rerun produced (4 cases), and the separation holds on the cold-only subscore. opus-5's 6/16 sits in the same tail, and that half of the reading is cross-family: opus-5 sat the Claude path's exam — prompt-only, no tools (Section 5.4) — so, assuming fewer affordances can only depress a score, read its observed score as a floor, pending v4.

Second, the grounding scan flagged nothing. On v3 it checked 105 numeric literals across 94 artifacts and flagged none; pooled across all three boards — two scan versions, the change disclosed in Section 5.1 — it checked 279 literals across 250 artifacts and still flagged none, the two grounding-trap cases of Section 2 included. The scan's sensitivity is what Section 3 describes and no more, so the flat claim this supports reads: on this suite, the binding check was test validation, not literal grounding — models fail by writing encodings whose tests or validation prove them wrong, not by inventing rates.

Third, speed and cost split by backend, and family rides the backend exactly: every OpenAI-family runner rode codex, every Anthropic-family runner rode the Claude CLI, on every board. The codex runners' median case ran 34–50 seconds on v3; fable's ran 266 seconds and emitted 458,159 output tokens, putting its suite cost at $33.02 against $5.66 for sol — 5.8× the cost at a lower gate-pass count on this board, though the two figures rest on different cost bases, which the annex labels. How much of the latency and token split reflects the models and how much the harness modes they ran under — an agentic workspace loop for codex, prompt-only generation for Claude — is exactly what Section 5.4 says v1–v3 cannot separate.

Table 5: Board v3 cost annex, API-equivalent; rows ordered by suite cost, and per pass = cost per gate pass. Codex-backend rows price recorded tokens at published July 2026 rates, cache reads at 10% of the input rate. The Claude CLI reports its own per-call cost, recorded verbatim; recomputing it from a rate table would mean inventing a rate for claude-opus-5, which has no published price in the reference the harness bundles. The Claude CLI also reports near-zero input tokens in its non-interactive print mode, and we take its recorded cost as pricing the full exchange.

runner	input tok	output tok	cache read	cost	per pass	basis
luna	826,676	43,734	310,528	$1.12	$0.14	rate table
terra	846,099	31,240	308,992	$2.66	$0.24	rate table
opus-5	36	152,826	774,613	$3.85	$0.64	CLI-reported
sol	897,438	33,026	356,608	$5.66	$0.40	rate table
gpt-5.5	898,057	37,576	344,576	$5.79	$0.39	rate table
fable	18	458,159	462,774	$33.02	$2.75	CLI-reported

5 What it takes to measure this honestly

5.1 Scores bind to the toolchain

Between board v2 (encoder 0.2.1380, 2026-07-25) and board v3 (0.2.1382, 2026-07-26), gate-pass counts moved: gpt-5.5 +4, opus-5 +4, fable +3, luna +3, terra 0, sol -1 — five of the six models on both boards, net +13. At case level the movement decomposes into 15 fail-to-pass flips against 5 pass-to-fail, plus three flips from fable's harness-killed v2 rows — those have a mechanical explanation, the timeout fix of Section 5.2. The remaining flips ran 15 to 5 toward passing — a skew symmetric sampling noise would produce with a two-sided exact binomial probability of about 0.04, and the flips cluster by model and case, which weakens even that. Three causes confound, and two runs cannot apportion them: sampling nondeterminism in the models, real toolchain change — the validator gained typed numeric occurrences and a locale profile between the boards (#1285), alongside

the timeout fix (#1284) — and server-side drift in the hosted models themselves. The fold's refusal to combine boards across encoder versions stopped looking like bookkeeping here and started doing its job.

Four things follow:

- **A board is one sample.** gpt-5.5 leads v3 at 15/16 with sol at 14/16 — one case apart, and gpt-5.5 itself moved four cases between boards. The order of ranks one and two is not a finding.
- **The extremes are findings.** The smallest gap separating the leaders from the tail (6 cases) exceeds every delta the rerun produced (4 cases).
- **A score is citable only with its encoder version.** This paper writes encoder versions next to every number for that reason, and the fold mechanically refuses to blend versions.
- **One non-extreme ranking result reproduced on every board where both models have clean rows: terra below sol.** terra — the production default the introduction discloses — scored 9, 11, 11 across the boards, the only model whose count did not move between the full-roster boards, while sol scored 14, 15, 14. No single board's gap clears the rerun yardstick on its own; the replication across all three boards is the finding. The July bake-off this benchmark formalizes had read the two as interchangeable on a smaller case set; what survives every board is that the larger, colder suite separates them.

5.2 A timeout that graded slowness as failure

Board v2's harness carried a hardcoded 600-second ceiling on the Claude encoder path. The codex path exposed configurable timeouts; the Claude path hardcoded one constant that nobody had reason to read. fable's median case ran 211 seconds on that board against the codex runners' 29–57 seconds — and that median understates fable's true case durations, because the harness killed 4 of its 16 cases with no artifact and recorded them at zero duration; over the 12 cases that completed, its median was 274 seconds. The fold scored each absent artifact as a compile and grounding miss. Among the completed cases, fable passed 9. The board record flagged the row as a floor the day it posted, and this paper does not cite it as a capability number anywhere.

The fix made the budgets configurable, recorded, and visible. On v3 the recorded policy reads: a uniform 3,600-second case budget over per-backend invocation budgets — Claude at 1,800 seconds of wall clock; codex at 600 seconds wall with idle detection for short sources and 1,800 for long ones. The budgets stay backend-shaped rather than identical — one more asymmetry for Section 5.4's ledger — but none of them bit a codex runner on v3, whose longest case ran 136 seconds. A case that exhausts its budget now records `timed_out`, renders as T on the grid, and drops out of the artifact denominators instead of masquerading as a bad encoding. Execution identity carries the whole timeout policy, so runs under different budgets refuse to fold, the same way runs from different encoders do.

Board v3 exercised the rebuilt timeout-and-finalization machinery on real artifacts: all six July runners completed the harness's finalize step with no operator recovery. On v2, two of six runners had needed it — the output of failed compiles embedded staging-directory paths, which differ per run, and the finalizer refused the mismatch. Recovery re-derived their results.json from the already-produced artifacts with the suite's revalidate command; no encoding or gate input changed, and the v2 board record discloses which runners. And v3's one timeout — fable's marriage_allowance_transfer, which exhausted its 3,600-second case budget — reads as exactly what it is.

5.3 The effort setting that did nothing

Every board above ran with a harness line that appeared to pin codex reasoning effort: `-c reasoning_effort="low"` on every codex invocation. `reasoning_effort` is not a recognized Codex configuration field — the real key is `model_reasoning_effort` — and Codex invoked without `--strict-config` accepts unknown keys silently. The line never did anything. Every codex runner on

every board ran at its receiver default — the *receiver* being whatever sits across the invocation, here the CLI and the service behind it — and the Claude path passes no effort flag at all, so those runners ran at their defaults too. A note posted to the tracking issue first reported the runs as pinned to low effort; receiver verification the same day corrected the record to inert. As a defect class this is worse than a wrong setting: the source reads as though the comparison is controlled.

A probe established what each knob does on one task: a nine-seat constraint-counting puzzle with a single integer answer, 5 samples per level on the codex arms and 3 on the Claude CLI — an *arm* being one (runner, effort) condition. (The frozen probe record's own header says "N=5 per level"; that holds only for the codex arm — an instrument misdescribing itself in a section about instruments misdescribing themselves.) On codex, the recognized key moves real resources on this probe: gpt-5.6-terra's median reasoning tokens rose from 145 at low to 2,756 at xhigh and median wall time from 20 to 53 seconds, though not monotonically — ultra's token median (1,685) fell below high's (2,243) while its wall time ran longest (106 seconds) with the widest spread (401–6,907 reasoning tokens). On the Claude CLI, `--effort` produced no monotone trend at 3 samples per level: claude-opus-5 returned the identical answer at every level, wall time held at 11–13 seconds, and the cost medians sat at $0.030–$0.033 everywhere except low ($0.051 — elevated at three samples, which ranged $0.031–$0.227). Two hypotheses fit that result and this probe cannot separate them: the model regulates its own reasoning depth, or the flag is inert in the CLI's print mode — the same defect class this section began with. A task the model answers identically at every level cannot detect added depth either way.

The design consequence holds under both hypotheses. Cross-family effort matching has no honest implementation on these receivers, so the design that survives is asymmetric and says so: sweep the codex arms at declared, receiver-verified levels; run the Claude arms at their receiver default and label them that way. Board v4 records requested and effective effort in execution identity, so effort-mismatched runs refuse to fold — the same mechanism that already refuses mixed toolchains.

5.4 Three backends, three exams

A read-only audit of the harness, posted in full to the tracking issue, found a deeper asymmetry: the three encoder paths did not administer the same exam. The codex path ran in a workspace and could shell-read the full context files. The Claude path ran prompt-only, with no tools. The OpenAI API path truncated each context file to a 6,000-character prefix, silently, and capped output at 16,384 tokens including reasoning, with partial artifacts eligible for scoring — that path produced no board row on v1–v3 (every frozen row rode codex or the Claude CLI), so its defects shaped no number here, but it sat in the harness as a live option. CLI versions went unrecorded, and version drift changes which flags a CLI even accepts; the reviewer's resolved identity went unrecorded the same way. The timeout budgets of Section 5.2 differ by backend too.

None of this shows on a leaderboard cell, and the two paths that produced rows shaped every score. The v1–v3 cross-family readings compare models on different evidence, and this paper flags each one where it appears — the first and third readings of Section 4, including fable's cost multiple — as weaker than the within-family readings.

The parity encoder closes the gap by construction rather than by calibration: every backend receives identical prompt bytes with all context inlined; the codex workspace starts empty; context overflow fails loudly as an infrastructure error instead of truncating silently; the harness rejects partial or truncated artifacts instead of scoring them; undeclared file access terminates the case; and the harness preflights and records CLI versions per row.

5.5 Case 04, flagged and retracted

All ten model-runs across boards v1 and v2 failed `income_tax_rate_bands` — nine as genuine attempts, plus fable's v2 row, which the 600-second ceiling killed before it could attempt — and the

v2 record flagged the case as a suspected suite defect: at zero for ten, a broken fixture explained the data at least as well as a hard provision. Board v3 answered — gpt-5.5 and fable both passed it — and we retracted the flag on the tracking issue the day the board posted. The failures had measured difficulty, not validity. The operating rule the episode set: unanimous failure justifies reading the artifacts, never disqualifying the case from a summary table.

5.6 Integrity notes

The archived boards carry three integrity notes, and the two excluded runs they concern are frozen in this paper's snapshot, marked excluded, so their numbers derive like every other.

opus-5's first v2 run returned 1/16 with nine cases dying on a session-limit error at a median of 0.9 seconds: its encoder and its reviewer drew on one subscription account, and the account hit its limit. We discarded that run as a measurement failure, split the accounts, and re-ran clean — zero quota errors, case durations of 10 to 465 seconds, and the executed model verified from traces; we published only the clean row. luna's v2 advisory-review coverage reached 8 of 16 after the reviewer's account hit its own limit mid-run; none of the four deterministic gates reads the reviewer, so its gate column stood. Both events taught the same operating rule: an encoder never shares an account with its own judge.

And fable ran alongside board v1, not only after it. Its first attempts died on harness infrastructure — a malformed MCP configuration that killed every Claude call instantly, then CLI authentication — and its completed 2026-07-24 run produced 16 case rows that the finalizer then refused: the nondeterministic-reviewer revalidation defect (#1280) later fixed. No results.json exists, so the run could never fold into board v1 — whose record had promised fable would join it. It is frozen here as excluded, and its rows corroborate Section 5.2: two cases killed at the 600-second ceiling, a 273-second median, and 8 of 16 gate passes among 13 artifacts — a floor under the same ceiling that shaped v2, published as capability neither then nor now.

6 Board v4, pre-registered

Board v4 runs when a parity encoder merges. The first attempt, PR #1304, closed unmerged on 2026-08-21 after main moved several hundred commits beneath it; its replacement, rebuilt on current main, was not yet open at this revision, and main still carries the inert effort key and the prompt-only Claude path. The committed design carries over:

- **Same sixteen cases, same corpus release, new encoder** — a new board by construction, folded by the same contract.
- **One exam.** Identical prompt bytes on every backend, context inlined, empty workspaces, overflow loud, partials rejected, undeclared access terminal; the harness preflights and records CLI versions and the reviewer's resolved identity per row (Section 5.4).
- **Effort as a recorded axis.** Codex arms swept at low, high, and ultra; each run records its requested level beside its reasoning-token count for post-hoc consistency checks — token counts separate extreme levels, not adjacent ones (Section 5.3). Claude arms run at their receiver default, disclosed as such. Effort-mismatched runs refuse to fold.
- **Repeated runs per arm on the pinned encoder.** The primary readout is the per-arm gate-pass mean with min–max range; the yardsticks of Section 5.1 govern any comparison claim. Single draws retire.
- **A scan positive control, and a real recall metric.** Before the board folds, a seeded canary artifact with a planted ungrounded literal must fail the grounding gate; and a source-relative coverage metric replaces the advisory column whose denominator is the artifact's own summary (Section 3). On the admission score that metric is a gate (Section 6.1).
- **Every run publishes.** The fold may exclude a run only for infrastructure failures of enumerated classes — quota exhaustion, authentication, harness crash — and an excluded run's results

freeze alongside the board, marked excluded, as this paper's snapshot already does for the runs in Section 5.6.

6.1 The admission score

A four-gate pass certifies a mergeable, self-consistent, grounded draft (Section 3). Production admission asks for more. Axiom's signed re-encode workflow admits a candidate only when the validator passes with policy proofs required and, by default, complete-source-unit checks required. Of the 106 signed runs that failed between 2026-09-24 and 2026-10-09, 49 ended at a complete-source-unit refusal, a check boards v1 to v3 did not run. A board that cannot fail a model where production fails it cannot say which model production should run. Board v4 reports a second score beside the four-gate headline.

The predicate. A candidate is *admitted* when the pinned encoder's deterministic admission check passes. That check is the validator pipeline configured as the signed apply path configures it: policy proofs required, complete source unit required, external oracles and model reviewers off, and the existing target's oracle contract supplied where a target exists. It returns two results, `compile` and `ci`, and both must pass. Every `ci` issue refuses; there is no advisory channel. Seven complete-source-unit categories sit inside `ci`: authoritative source, structure, deferral, formula output, source-explicit conditions, tests and numeric recall.

Numeric recall is a gate. The numeric-recall category is the source-relative metric promised above. Its denominator is the numeric occurrences of the authoritative source unit after the validator's exemptions, not those of the artifact's own summary, and one unrepresented occurrence refuses. The board reports covered, total and missing occurrences for every candidate. A source with no numeric obligation passes and shows no percentage.

The pin. The predicate is the code at one commit of axiom-encode main. That mirrors production: a signed run installs the encoder from the commit it was dispatched at, and keeps that commit however long it waits for approval. The pinned commit, its package version, and the rules-engine and compose identities post to the tracking issue before the first run. The same package computes the four-gate result on the same candidates, in separate fields. An admission refusal never overwrites a four-gate result.

What is scored. Production does not admit first drafts. It runs a bounded loop: deterministic repairs, then validator feedback to the model, for at most four attempts. The primary readout is *chain admission*: whether a fresh chain, run under the pinned commit's own repair, feedback and attempt policy, ends admitted. First-attempt admission reports beside it. Corpus materialization, authorized refs, ownership manifests, attestation and signing are prerequisites of the transaction, not properties of the candidate. They are packaged before any run, and a failure there is an enumerated infrastructure exclusion, published with the board.

A held-out admission suite. The sixteen UK cases keep the headline. The admission score also runs on a second suite: the 37 citations (15 federal, 19 state and District of Columbia, 3 Canadian) at which those failed signed runs ended on a check intrinsic to the candidate. The list freezes as a manifest, with its sha256 in this repository, before any v4 run and before any prompt, gate or routing change made in response to it. A citation whose signed source cannot be rematerialized drops before the first run, and the dropped list publishes. This suite is a failure census. It measures rescue under fixed gates. It cannot estimate an admission rate over all work.

Gates move between the census and the pin. These citations failed under the encoder of their day, and some refusals were validator faults since fixed. One Oklahoma statute was refused for not representing 162, which is a session-law chapter number in its history note. The score is taken at the pin, never at the census. A citation stays in the suite whether or not a later gate fix would now admit it, and the gate changes merged between the census and the pin are listed with the pin.

Arms. Every arm runs the codex backend, so the arms sit one exam by construction. Production's signed workflow generates through the API backend instead. The admission check is the same code, but the generation exam is the codex backend's (Section 5.4), so the baseline arm is production's routing on another backend, not production itself. An arm is a setting of the encoder's own retry options, with the budget held at four attempts. The arms are the production default (two attempts on gpt-6-luna, then two on gpt-6-sol) and single-model chains on gpt-6-luna, gpt-6-sol and gpt-6.1-sol. Cross-family arms join when the parity encoder merges. The repeat count posts with the roster.

Reading it. A citation *splits* when at least one arm admits it in some chain and at least one arm admits it in none. When no arm admits a citation and every chain ends at the same refusal category, the finding concerns the prompt or the gate, and it is filed as an encoder issue. A routing claim across the suite needs one arm to admit more citations than another by more than either arm's own range across repeats (Section 5.1).

What follows from it. One decision is fixed before the data. If a quarter or more of the scored citations split, a forward routing trial follows: arms assigned at dispatch, in production, on work not selected for having failed. If fewer split, no routing trial follows, and the effort goes to aligning the encoder with its gates. Either way, this board alone supports no claim of production gain. That claim needs the forward trial: admitted output per unit of time and spend, failures included.

This pre-registration binds through the paper's public git history. The run count per arm and the exact roster stay open until launch and will post to the tracking issue before the first run starts. v4 is the first board whose cross-family rows sit the same exam as its within-family rows; effort remains a disclosed asymmetric axis (Section 5.3). This paper refreezes with it: new snapshot, re-derived numbers, same standing rule that board statistics appear only as derivations from frozen artifacts.

7 Limitations

Sixteen cases sample one jurisdiction on one corpus release; the suite measures UK statutory encoding on this release, not encoding in general, and the sixteen cluster into difficulty groups rather than sixteen exchangeable trials. Whole classes of statutory difficulty go unsampled: definitional and interpretive webs, commencement and transitional mechanics, devolved variation (a production income-tax encoding must handle Scottish and Welsh rates; no case here requires it), the secondary legislation where much benefit-system complexity lives (the universal credit case encodes the primary-legislation skeleton, not the 2013 regulations), and case-law gloss. The gates certify what Section 3 states — a mergeable, self-consistent, grounded draft — and the companion tests are the graded model's own, so the test gate's stringency is endogenous to the model. The oracle column that would cross-check computed results against PolicyEngine UK stays empty until that runtime path is verified for eval cases. Each (model, board) cell is a single run, so within-toolchain variance stays unmeasured until v4's repeated runs — and hosted models can drift server-side between boards, a cause the toolchain pin cannot hold fixed. Cold workspaces prevent copying at inference time, not memorization: most of these provisions are famous, their existing encodings are public, and only one case postdates training data. Costs are API-equivalents priced from recorded tokens for the codex runners and CLI-reported figures for the Claude runners. The runners are CLI products, not bare models — each brings its own system prompt, tool behavior, and retry policy — and boards v1–v3 did not equalize them (Section 5.4).

8 Reproducibility

Every board statistic, probe value, and integrity-note number in this paper derives in code from the frozen snapshot — the boards' `results.json` files (one per runner per board: 4+6+6), the two excluded runs of Section 5.6, the sixteen per-case workspace context manifests, the suite manifest as the v3 encoder commit pinned it, and the effort-probe record — each file sha256-checked against

the snapshot manifest at load. The derivation module re-implements the deterministic pieces of the `eval-board` fold, verifies its derived headlines against the board records posted to the tracking issue, and refuses to render the manuscript on any mismatch; this render passed that check (verified against published records: 16 runner gate counts, medians, v3 artifacts/timeouts/costs/coverage, the full probe tables, and the context-manifest binding). A few numbers are harness facts rather than run outputs — the 600-second v2 ceiling appears in the killed rows' own error text, while the OpenAI path's 6,000-character and 16,384-token constants live in the audited harness source — and the paper cites those to the audit rather than deriving them.

The snapshot attests content by hash. The runs' evidence signing and the corpus release's key verification happened rig-side at run time; what the repository lets a third party re-check is the hashes, plus every derivation in this paper. The freeze script reads a private rig directory — per-case traces, generated encodings, and reviewer verdicts are hashed into the frozen rows but not published — so run-level artifacts reproduce by hash comparison, not by re-execution.

axiom-encode holds the suite manifest, the harness, and the fold; the boards and their records live on tracking issue #1189, and PR #1304 holds the parity-encoder design its replacement inherits. The current board, the suite, and this paper live at encodebench.org; the paper's source, snapshot, and derivation module live in the site repository under `paper/`, and rendering it takes Quarto (1.9), a TeX distribution, and the pinned Python environment committed beside it.

References

Chen, Mark, Jerry Tworek, Heewoo Jun, et al. 2021. *Evaluating Large Language Models Trained on Code*. https://arxiv.org/abs/2107.03374.

Ghenis, Max. 2026. *PolicyBench: Benchmarking No-Tool Tax-and-Benefit Estimation in Frontier Language Models*. https://policybench.org/paper.

Guha, Neel, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, et al. 2023. "LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models." *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. https://arxiv.org/abs/2308.11462.

Holzenberger, Nils, Andrew Blair-Stanek, and Benjamin Van Durme. 2020. *A Dataset for Statutory Reasoning in Tax Law Entailment and Question Answering*. https://arxiv.org/abs/2005.05257.

Janatian, Samyar, Hannes Westermann, Jinzhe Tan, Jaromir Savelka, and Karim Benyekhlef. 2023. "From Text to Structure: Using Large Language Models to Support the Development of Legal Expert Systems." *Legal Knowledge and Information Systems (JURIX)*. https://arxiv.org/abs/2311.04911.

Jimenez, Carlos E., John Yang, Alexander Wettig, et al. 2024. "SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?" *International Conference on Learning Representations*. https://arxiv.org/abs/2310.06770.

Lorenzo, Gabriele, Aldo Pietromatera, and Nils Holzenberger. 2025. "Translating Tax Law to Code with LLMs: A Benchmark and Evaluation Framework." *Proceedings of the Natural Legal Language Processing Workshop*, 31–47. https://aclanthology.org/2025.nllp-1.4/.

Mérigoux, Denis, Nicolas Chataing, and Jonathan Protzenko. 2021. "Catala: A Programming Language for the Law." *Proceedings of the ACM on Programming Languages* 5 (ICFP). https://doi.org/10.1145/3473582.

Mohun, James, and Alex Roberts. 2020. *Cracking the Code: Rulemaking for Humans and Machines*. OECD Working Papers on Public Governance No. 42. OECD Observatory of Public Sector Innovation. https://doi.org/10.1787/3afe6ba5-en.

OpenFisca contributors. 2026. *OpenFisca: Open-Source Engine to Write Rules as Code*. https://openfisca.org.